KIET DEEMED TO BE UNIVERSITY, GHAZIABAD

Software Requirements Specification

for

AyurSutra

Cloud-Based Ayurvedic Practice Management & AI-Assisted Personalized Wellness Platform

Prepared by: AyurSutra Project Team

KIET Deemed to be University, Ghaziabad

S. No.	Name	Role	Registration No.
1	**Vanshika**	Team Leader	202401100200338
2	Simmi Kumari	Team Member	202401100200305
3	Tejaswini Mishra	Team Member	202401100200324
4	Vrinda Aggarwal	Team Member	202401100200355

Table of Contents

Table of Contents ii
Revision History iii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 2
1.4 Product Scope 2
1.5 References 3
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 4
2.3 User Classes and Characteristics 4
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 5
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements 7
3.1 User Interfaces 7
3.2 Hardware Interfaces 7
3.3 Software Interfaces 8
3.4 Communications Interfaces 9
4. System Features 10
4.1 Authentication and Authorization 10
4.2 Admin Management 11
4.3 Patient and Doctor Profile Management 11
4.4 Panchakarma Therapy Management 12
4.5 Appointment Management 13
4.6 Nutrition Management and Analysis 13
4.7 AI-Assisted Personalized Diet Recommendation 14
4.8 Progress Tracking, Notifications and Reports 15
5. Other Nonfunctional Requirements 16
5.1 Performance Requirements 16
5.2 Safety Requirements 16
5.3 Security Requirements 16
5.4 Software Quality Attributes 16
5.5 Business Rules 17
6. Other Requirements 17
Appendix A: Glossary 18
Appendix B: Analysis Models 19
Appendix C: To Be Determined List 22

Revision History

Name	Date	Reason For Changes	Version
AyurSutra Project Team	07-Sep-2026	Initial SRS drafted in project-specific format	0.1
AyurSutra Project Team	07-Sep-2026	Reformatted into standard IEEE/KIET SRS template as required by project supervisor	1.0

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for AyurSutra, a responsive web-based Ayurvedic Practice Management and Personalized Wellness Platform. It establishes a common understanding of the system's purpose, scope, users, behaviour, interfaces, quality attributes, data requirements and system models.

The document guides requirements analysis, UI/UX design, development, testing, academic demonstration and future maintenance of the project. Requirements are expressed using normative language ("shall") and unique identifiers (REQ-xx, NFR-xx) so that they can be traced to implementation and test cases throughout the semester project.

Objectives

- Define the system boundary and intended capabilities.
- Define role-based access for Admin, Doctor and Patient.
- Specify the complete Panchakarma, appointment, nutrition and AI diet workflow.
- Provide testable functional and non-functional requirements.
- Provide a common basis for architecture, database and UI/UX design.

1.2 Document Conventions

This document follows a standard IEEE 830 / Karl Wiegers-style SRS structure, adapted for an academic semester project at KIET Deemed to be University.

- The keyword "shall" indicates a mandatory, testable requirement.
- The keyword "should" indicates a recommended but non-mandatory characteristic.
- Functional requirements are identified using the prefix REQ- followed by a two-digit sequence number (e.g., REQ-01).
- Non-functional requirements are identified using the prefix NFR- followed by a two-digit sequence number (e.g., NFR-01).
- Headings and sub-headings follow the numbering scheme of this template; body text uses 12-point Calibri/Times New Roman as per KIET formatting guidelines.
- Unless stated otherwise, every requirement carries a default priority of "Medium"; requirements explicitly marked "High" are considered essential for the Minimum Viable Product (MVP).

1.3 Intended Audience and Reading Suggestions

This SRS is intended for the project team, project supervisor, UI/UX designers, frontend and backend developers, AI/ML developers, database developers and testers.

Recommended Reading Order

- Section 1 — purpose, scope and document conventions.
- Section 2 — product context, users, environment, constraints and dependencies.
- Section 3 — external interface requirements.
- Section 4 — detailed system features and functional requirements.
- Section 5 — non-functional requirements.
- Section 6 — other requirements.
- Appendices — glossary, analysis models/diagrams and the to-be-determined list.

For implementation, each REQ/NFR identifier should become a development task and a corresponding test case. This provides requirements traceability throughout the semester project.

1.4 Product Scope

AyurSutra provides a unified platform for managing the operational and patient-care workflows of an Ayurvedic practice. The system has exactly three roles: Admin, Doctor and Patient. There is no Dietician role. AI generates personalized diet recommendations, while the Doctor reviews and approves or modifies them before patient visibility.

In Scope

- Role selection and authentication
- Role-based dashboards and authorization
- Doctor and patient account management
- Patient profiles and structured Ayurvedic assessment
- Panchakarma therapy catalog, treatment plans and session scheduling
- Appointment calendar and status management
- Nutrition data and nutrition analysis
- AI-assisted personalized diet recommendation
- Doctor review, modification and approval
- Patient progress tracking
- Notifications, reports and analytics

Out of Scope

- Dietician accounts or a separate dietician workflow
- Autonomous medical diagnosis or autonomous clinical decision-making
- Emergency care or hospital information-system functions
- Laboratory, pharmacy and insurance integrations
- Specialized wearable/biometric hardware in the semester-project version
- Publishing unapproved AI recommendations to Patients
- Hard-coded clinical protocols that cannot be configured

1.5 References

- ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
- IEEE 830-style Software Requirements Specification organization, adapted for a modern academic project.
- KIET Deemed to be University academic/semester-project documentation and applicable submission guidelines.
- Official documentation for React.js, Node.js, Express.js, MongoDB Atlas, Python, FastAPI and AWS, according to the versions selected during implementation.
- Project-specific AyurSutra requirements and workflow agreed by the project team.

This document uses an IEEE-style requirements structure. It is an academic project specification and does not claim formal certification against an IEEE/ISO standard.

2. Overall Description

2.1 Product Perspective

AyurSutra is a modular web application for an Ayurvedic clinic or practice. It combines operational management and patient wellness workflows in a single application. It is a new, self-contained system and is not a replacement for or component of any existing hospital information system.

System Context

- Admin manages system-level entities and configuration.
- Doctor manages patient care workflows, Panchakarma, appointments, nutrition analysis and AI recommendation review.
- Patient accesses only authorized personal information and approved plans.
- The backend controls authorization and mediates database and AI-service interactions.

At a high level, Admin, Doctor and Patient users interact with a React.js web application, which communicates with a Node.js/Express.js backend over REST APIs. The backend persists data in MongoDB Atlas and delegates diet-recommendation generation to a separate Python/FastAPI AI service. The complete application stack is deployed on AWS.

2.2 Product Functions

The table below summarizes the major functional modules of AyurSutra at a high level; each module is elaborated as a System Feature in Section 4.

Module	High-Level Function
Authentication	Role selection, login, authorization and logout.
Admin	Manage doctors, patients, therapies, appointments, analytics and reports.
Patient Management	Patient profiles, assessment, relevant history and notes.
Panchakarma	Therapy catalog, plans, sessions, schedules and status tracking.
Appointments	Calendar, creation, rescheduling, cancellation and completion.
Nutrition	Nutrition records, nutrient summaries and comparison/analysis.
AI Diet	Personalized diet recommendation generation.
Doctor Review	Modify, regenerate, approve or save AI recommendation as draft.
Progress	Chronological progress records and configurable wellness metrics.
Patient Portal	Appointments, therapies, approved diet, progress and notifications.
Reports	Operational and role-appropriate reports and analytics.

2.3 User Classes and Characteristics

User Class	Characteristics	Primary Access
Admin	System/operations manager; manages accounts and configuration.	System management, analytics, reports.
Doctor	Primary practitioner; needs rapid access to patient-care workflows.	Patients, assessments, Panchakarma, appointments, nutrition, AI review and progress.
Patient	End user; needs simple, readable access to personal care information.	Own appointments, therapies, approved diet, progress and notifications.

Role Boundaries

- Admin does not perform the Doctor's clinical workflow.
- Doctor reviews AI-generated diet recommendations before patient visibility.
- Patient cannot directly edit Doctor-created treatment plans.
- AI has no login role and does not replace the Doctor.

2.4 Operating Environment

Area	Environment / Constraint
Client	Modern browser such as Chrome, Edge, Firefox or Safari.
Frontend	React.js responsive single-page web application.
Backend	Node.js runtime with Express.js REST API.
Database	MongoDB Atlas.
AI Service	Python runtime with FastAPI and selected recommendation/ML libraries.
Deployment	AWS-hosted application/services according to the final deployment design.
Network	Internet/network connection required for normal cloud operation.
Source Control	Git/GitHub; secrets kept outside source code.

Practical Development Environment

- Developer workstation with a modern CPU and at least 8 GB RAM recommended.
- Modern code editor/IDE and Git client.
- Browser developer tools for frontend testing.
- Internet access for cloud database, APIs and deployment.

2.5 Design and Implementation Constraints

- Exactly three application roles: Admin, Doctor and Patient.
- No Dietician account type.
- MongoDB Atlas is the planned application database.
- AWS is primarily the deployment platform, not the core database/storage layer.
- AI is separated as a Python/FastAPI service.
- Patient information is protected through authentication and authorization.
- AI diet recommendations require Doctor review before Patient visibility.
- Therapy information should be configurable rather than hard-coded as clinical protocol.
- The project shall remain feasible within a semester-project timeline.

2.6 User Documentation

This SRS is the primary technical reference document for the project team and supervisor. In addition to this document, the following user-facing documentation is planned to accompany the delivered system:

- A short role-wise user guide (Admin, Doctor and Patient) covering login and core workflows, delivered as part of the final project report.
- In-application guidance such as field-level hints, empty-state messages and confirmation prompts, in place of a separate printed manual (see Section 3.1 and Section 4).
- A README file accompanying the source-code repository describing setup, configuration and environment variables.

As this is a semester project rather than a commercially distributed product, no separate installation manual, tutorial or context-sensitive online help beyond the above is planned at this stage; any such artifact required by the supervisor will be produced as an additional deliverable and referenced here.

2.7 Assumptions and Dependencies

Assumptions

- Authorized Doctors are responsible for clinical decisions.
- Admins maintain user accounts and configurable therapy records.
- Patients use authenticated accounts.
- A suitable nutrition/food dataset is available for the academic prototype.
- AI recommendations are assistive outputs, not autonomous prescriptions.

Dependencies

- MongoDB Atlas availability
- AWS deployment environment
- FastAPI AI service availability
- Nutrition/food dataset quality
- Selected third-party libraries and runtime versions
- Stable network connectivity

3. External Interface Requirements

This section specifies the interfaces between AyurSutra and its users, hardware, software components and communication channels. The frontend shall not directly access MongoDB Atlas or the protected AI service; the backend shall mediate these interactions.

3.1 User Interfaces

- Login shall clearly present Admin, Doctor and Patient options.
- The application shall provide role-specific sidebars and dashboards.
- Doctor screens shall prioritize patient search, patient profile, appointments, Panchakarma and AI review.
- Patient screens shall use simplified cards, timelines and readable status information.
- The application shall use consistent typography, spacing, buttons, cards, forms, tables and status badges.
- Calendar interfaces shall support day, week and month views where applicable.
- Responsive layouts shall adapt to desktop, tablet and mobile screens.
- Accessibility-oriented practices such as visible labels, readable contrast and non-color-only status communication shall be followed.

Representative UI Messages

State	Example Message
Loading	Loading patient information...
Empty	No upcoming appointments found.
Success	Panchakarma plan created successfully.
AI Pending	AI recommendation generated and is awaiting Doctor review.
AI Approved	Diet plan approved and is now available to the Patient.
Error	We could not complete this operation. Please try again.

3.2 Hardware Interfaces

The current semester-project version does not require specialized clinical hardware.

Hardware	Interface Requirement
Desktop/Laptop	Standard keyboard, mouse/trackpad, display and modern browser.
Tablet	Touch-enabled responsive browser interface.
Smartphone	Responsive mobile browser interface.
Specialized medical devices	Not required in current scope; future integration only.
Wearables/Biometrics	Not required in current scope; future enhancement.

The system does not directly control medical equipment. Any future hardware integration requires a separate interface specification, security review and validation.

3.3 Software Interfaces

Software Component	Interface / Responsibility
React.js	Renders UI, manages client-side navigation/state and calls backend APIs.
React Router	Provides protected and role-specific frontend routes.
Axios / HTTP client	Sends requests to REST APIs and receives JSON responses.
Node.js + Express.js	Provides authentication, authorization, business logic and REST endpoints.
MongoDB Atlas	Persistent storage for application records.
FastAPI	Exposes the AI recommendation service to the backend.
AI/ML libraries	Implement recommendation/data-processing logic within the AI service.
AWS	Provides deployment environment for application/services.

API Interface Summary

The backend shall expose REST-style endpoints grouped by module, summarized below. Exact route names may be finalized during implementation while preserving the required behaviour (see Appendix C).

API Group	Representative Operations	Consumers
Authentication	login, logout, token/session validation	All roles
Doctors	create, list, view, update, activate/deactivate	Admin
Patients	create, search, view, update	Doctor/Admin
Assessments	create, view, update	Doctor
Therapies	create, list, update, activate/deactivate	Admin
Therapy Plans	create, view, update, schedule sessions	Doctor
Appointments	create, list, reschedule, cancel, complete	Doctor/Admin; Patient view
Nutrition	store/retrieve records and analysis inputs	Doctor

API Group	Representative Operations	Consumers
AI Recommendations	generate, retrieve, modify, approve	Doctor/backend
Progress	create and retrieve records	Doctor; Patient view
Notifications	create, retrieve, mark read	System/users
Reports	generate, retrieve, export	Admin/Doctor

3.4 Communications Interfaces

Communication Path	Protocol / Format	Security Expectation
Browser → Backend	HTTPS + REST + JSON	HTTPS in deployed environment; authenticated requests.
Backend → MongoDB Atlas	MongoDB driver/ODM protocol	Credentials protected; no client-side direct access.
Backend → FastAPI	HTTPS/HTTP service API + JSON	Protected service-to-service access.
Deployment/Source Control	Git-based workflow	Secrets excluded from repository.

Communication Rules

- JSON shall be the default API payload format.
- HTTP status codes shall communicate success/failure categories.
- Authentication and authorization failures shall not expose sensitive details.
- Production secrets shall be stored using environment/deployment configuration.
- Frontend shall handle network timeouts and service failures gracefully.

AI Service Interface

The AI service shall be a separately deployable Python/FastAPI service. The Node.js backend shall act as the controlled integration point.

- The frontend shall not directly call the protected AI service.
- The backend shall validate Doctor authorization before requesting a recommendation.
- The backend shall send only relevant authorized inputs.
- The AI service shall return structured JSON-compatible recommendation data.
- The backend shall validate the AI response before storing it.
- AI output shall initially be stored as Draft/Pending Review.
- AI service failure shall not publish a partial or unapproved diet plan.
- The service interface shall be versionable so recommendation logic can evolve independently.

Interface Error Handling

- Unauthorized requests shall be rejected by the backend.
- Invalid form submissions shall show field-level or actionable validation feedback.
- Network/API failures shall show a user-friendly error state.
- AI generation failures shall show a retryable error state.
- Loading indicators shall be displayed for asynchronous operations.
- Empty results shall show contextual empty-state messages.
- Successful operations shall provide confirmation feedback.
- Destructive actions shall require confirmation.
- Internal stack traces, database errors and secret values shall not be exposed to end users.

4. System Features

This section decomposes AyurSutra into system features (functional modules). Each feature states its description and priority, representative stimulus/response sequences, and the detailed functional requirements that implement it.

4.1 Authentication and Authorization

4.1.1 Description and Priority

Provides role-based login for Admin, Doctor and Patient, and enforces authorization on every protected resource. This feature is foundational — no other feature is reachable without it.

Priority: High

4.1.2 Stimulus/Response Sequences

- User opens the application → system presents Admin, Doctor and Patient login options.
- User selects a role and submits valid credentials → system authenticates and redirects to the corresponding dashboard.
- User submits credentials for a role that does not match their authorized account role → system rejects access with an actionable error message.
- Authenticated user selects logout → system securely terminates the session.

4.1.3 Functional Requirements

REQ-01: The system shall present Admin Login, Doctor Login and Patient Login options.

REQ-02: The system shall authenticate registered credentials after role selection.

REQ-03: The system shall reject access when the selected role does not match the authorized account role.

REQ-04: The system shall redirect authenticated users to the corresponding role-specific dashboard.

REQ-05: The system shall enforce authorization on protected backend APIs.

REQ-06: The system shall hash passwords before storage.

REQ-07: The system shall provide secure logout.

4.2 Admin Management

4.2.1 Description and Priority

Allows the Admin to manage Doctor and Patient accounts at the system level, maintain the configurable therapy catalog, and view organization-wide appointments, analytics and reports.

Priority: High

4.2.2 Stimulus/Response Sequences

- Admin logs in → system shows the Admin dashboard with account, therapy and analytics options.
- Admin creates or deactivates a Doctor account → system updates the account and reflects the change in Doctor login availability.
- Admin adds a new therapy catalog entry → system makes the therapy available for Doctors to include in treatment plans.
- Admin opens the appointment overview → system displays organization-wide appointment status and analytics.

4.2.3 Functional Requirements

REQ-08: The system shall allow Admins to manage Doctor accounts.

REQ-09: The system shall allow Admins to manage Patient accounts at the system-management level.

REQ-10: The system shall allow Admins to manage configurable therapy catalog entries.

REQ-11: The system shall provide Admin appointment overview, analytics and reports.

4.3 Patient and Doctor Profile Management

4.3.1 Description and Priority

Enables Doctors to search for and create Patient profiles, and to record structured Ayurvedic assessments, relevant history and notes. Patients are restricted to viewing only their own authorized information.

Priority: High

4.3.2 Stimulus/Response Sequences

- Doctor searches for an existing Patient or creates a new Patient profile → system returns matching results or creates a new record.
- Doctor opens a Patient profile → system displays Overview, Ayurvedic Assessment, Medical History, Panchakarma, Appointments, Nutrition, AI Diet Plan, Progress and Notes tabs.
- Doctor records or updates the Ayurvedic assessment, allergies, medications or notes → system saves the update and shows success feedback.

4.3.3 Functional Requirements

REQ-12: The system shall allow Doctors to search and create Patient profiles.

REQ-13: The system shall provide a Patient Profile containing Overview, Ayurvedic Assessment, Medical History, Panchakarma, Appointments, Nutrition, AI Diet Plan, Progress and Notes.

REQ-14: The system shall allow Doctors to record structured Ayurvedic assessment information.

REQ-15: The system shall allow Doctors to record relevant history, allergies/intolerances, medications/supplements and practitioner notes where applicable.

REQ-16: The system shall restrict Patients to their own authorized information.

4.4 Panchakarma Therapy Management

4.4.1 Description and Priority

Provides a configurable Panchakarma therapy catalog and allows Doctors to build, schedule and track therapy plans and individual sessions for a Patient.

Priority: High

4.4.2 Stimulus/Response Sequences

- Doctor selects a therapy from the catalog and a Patient → system creates a draft therapy plan.
- Doctor defines plan dates, session frequency, duration and notes → system saves the plan and generates the session schedule.
- Doctor marks a session complete or reschedules it → system updates the session status and the overall plan status accordingly.
- Plan reaches its final session → system transitions the plan status to Completed.

4.4.3 Functional Requirements

REQ-17: The system shall provide a configurable Panchakarma therapy catalog.

REQ-18: The system shall allow Doctors to create a therapy plan for a selected Patient.

REQ-19: The system shall allow Doctors to define plan dates, sessions, frequency, duration and notes.

REQ-20: The system shall allow Doctors to schedule individual therapy sessions.

REQ-21: The system shall support Upcoming, In Progress, Completed, Paused and Cancelled therapy plan statuses.

REQ-22: The system shall allow Doctors to complete, reschedule and update therapy sessions.

4.5 Appointment Management

4.5.1 Description and Priority

Provides calendar-based appointment creation, viewing and lifecycle management for Doctors, Admins and Patients.

Priority: High

4.5.2 Stimulus/Response Sequences

- Doctor creates an appointment for a Patient and selects an appointment type → system adds it to the calendar with status Scheduled.
- Doctor or Admin switches between day, week and month calendar views → system renders the appropriate view.
- Authorized user reschedules or cancels an appointment → system updates the status and notifies affected users.
- Appointment time passes without check-in → system marks the appointment No-show.

4.5.3 Functional Requirements

REQ-23: The system shall allow Doctors to create appointments for Patients.

REQ-24: The system shall support Consultation, Follow-up, Panchakarma Session and configurable other appointment types.

REQ-25: The system shall provide day, week and month calendar views.

REQ-26: The system shall support Scheduled, Confirmed, Completed, Cancelled and No-show statuses.

REQ-27: The system shall allow authorized users to reschedule or cancel appointments according to permissions.

4.6 Nutrition Management and Analysis

4.6.1 Description and Priority

Stores nutrition attributes for food items and compares them against a Patient's dietary information to support the AI diet recommendation workflow.

Priority: Medium

4.6.2 Stimulus/Response Sequences

- Doctor opens the Nutrition tab for a Patient → system displays stored nutrition records and available nutrient data.
- Doctor requests a nutrition comparison → system computes and presents readable results, indicators or charts.
- System detects insufficient nutrition data for a conclusive comparison → system presents a contextual empty/insufficient-data state rather than a diagnosis.

4.6.3 Functional Requirements

REQ-28: The system shall store or access nutrition attributes for food items used in the recommendation workflow.

REQ-29: The system shall support calories, protein, carbohydrates, fats and fiber where available.

REQ-30: The system shall compare available nutrition requirements with planned or recorded dietary information.

REQ-31: The system shall present nutrition results using readable values, indicators or charts.

REQ-32: The system shall not use nutrition analysis alone to diagnose medical conditions.

4.7 AI-Assisted Personalized Diet Recommendation

4.7.1 Description and Priority

Generates a personalized diet recommendation for a Patient using the FastAPI AI service, and requires Doctor review and approval before the recommendation is visible to the Patient. This is the central differentiating feature of AyurSutra and directly implements the system's core safety rule that no unapproved AI output may reach a Patient.

Priority: High

4.7.2 Stimulus/Response Sequences

- Doctor initiates an AI diet recommendation for a selected Patient → backend collects relevant dietary, lifestyle, assessment and nutrition inputs and invokes the FastAPI AI service.
- AI service returns a structured recommendation → backend stores it with status Pending Review and notifies the Doctor.
- Doctor reviews the recommendation and chooses to modify, regenerate, approve or save it as draft → system updates the recommendation status accordingly.
- Doctor approves the recommendation → system marks it Approved, notifies the Patient, and displays the final diet plan on the Patient's dashboard.
- Patient attempts to view a recommendation that has not been approved → system does not expose it.

4.7.3 Functional Requirements

REQ-33: The system shall allow an authorized Doctor to initiate an AI diet recommendation for a selected Patient.

REQ-34: The system shall collect relevant dietary, lifestyle, assessment and nutrition inputs required by the recommendation workflow.

REQ-35: The backend shall send authorized inputs to the FastAPI AI service.

REQ-36: The AI service shall return a structured personalized diet recommendation.

REQ-37: The recommendation shall support meal sections and a nutrition summary where sufficient data is available.

REQ-38: The system shall mark AI-generated output as requiring Doctor review.

REQ-39: The system shall support Generated, Pending Review, Modified, Approved and Draft statuses.

REQ-40: The Doctor shall be able to approve, modify, regenerate or save an AI recommendation as draft.

REQ-41: The system shall not expose an unapproved AI recommendation to a Patient.

REQ-42: The system shall display the final approved diet plan to the Patient.

4.8 Progress Tracking, Notifications and Reports

4.8.1 Description and Priority

Maintains a chronological record of Patient progress, delivers in-application notifications for time-sensitive events, and provides role-appropriate reports, together with consistent system feedback (loading, empty, error and success states) across the application.

Priority: Medium

4.8.2 Stimulus/Response Sequences

- Doctor records a progress entry for a Patient after a session or review → system appends it to the Patient's progress timeline.
- An approved diet plan becomes available, or an appointment/therapy session is upcoming → system raises an in-application notification for the relevant user.
- User marks a notification as read → system updates its read status.
- Admin or Doctor requests a report → system generates and displays role-appropriate analytics.
- Any asynchronous operation is in progress, empty, fails, or succeeds → system shows the corresponding loading, empty, error or success feedback; destructive actions request confirmation first.

4.8.3 Functional Requirements

REQ-43: The system shall allow Doctors to create Patient progress records.

REQ-44: The system shall maintain a chronological progress timeline.

REQ-45: The system shall support configurable wellness metrics and therapy completion information.

REQ-46: The system shall display appropriate progress information to the Patient.

REQ-47: The system shall provide in-application notifications for relevant appointments and therapy sessions.

REQ-48: The system shall notify Patients when an approved diet plan becomes available.

REQ-49: The system shall allow notification items to be marked as read.

REQ-50: The system shall provide role-appropriate reports and analytics.

REQ-51: The system shall display loading states for asynchronous operations.

REQ-52: The system shall display meaningful empty states when no records are available.

REQ-53: The system shall display actionable error messages when operations fail.

REQ-54: The system shall display success feedback after important operations.

REQ-55: The system shall request confirmation before destructive actions.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

NFR-01: Standard dashboard and CRUD operations should provide a visible response within approximately 2 seconds under expected project workload and stable network conditions.

NFR-02: The application shall show progress feedback for long-running AI operations.

NFR-03: Common patient and appointment searches should use suitable database indexes or query strategies.

5.2 Safety Requirements

AyurSutra is an academic prototype and is not certified as a medical device. The following safety-oriented requirements reduce the risk of the system being mistaken for, or misused as, an autonomous source of clinical advice.

NFR-16: The system shall not present AI-generated or nutrition-analysis output as a medical diagnosis or as a substitute for professional clinical judgement.

NFR-17: An AI-generated diet recommendation shall never reach a Patient without explicit Doctor approval (see REQ-38, REQ-41).

NFR-18: The system shall retain an auditable record of who created, modified and approved each therapy plan, appointment and diet recommendation.

NFR-19: In the event of AI-service failure, the system shall fail safely by withholding output rather than publishing partial or unverified recommendations (see REQ-41).

5.3 Security Requirements

NFR-04: The system shall enforce role-based authorization on backend resources.

NFR-05: Passwords shall never be stored in plaintext.

NFR-06: Secrets shall be stored in environment configuration and excluded from source control.

NFR-07: Deployed client-server communication shall use HTTPS.

NFR-08: The backend shall validate authorization for every protected resource request.

NFR-09: Patient information shall be accessible only to authorized users.

5.4 Software Quality Attributes

Reliability

NFR-10: The system shall handle database, API and AI-service failures gracefully.

NFR-11: The system shall preserve saved records after normal session termination.

Usability

NFR-12: The UI shall use consistent terminology and navigation.

Maintainability

NFR-14: The frontend shall use reusable UI components.

NFR-15: The AI service shall remain separately maintainable and replaceable.

5.5 Business Rules

- The system supports exactly three application roles — Admin, Doctor and Patient; there is no Dietician role.
- An Admin does not perform the Doctor's clinical workflow, and a Patient cannot directly edit Doctor-created treatment plans.
- AI has no login role and does not replace the Doctor; every AI diet recommendation must be reviewed by a Doctor before it becomes visible to a Patient.
- Therapy catalog entries are configurable business data maintained by the Admin, not hard-coded clinical protocol.
- MongoDB Atlas is the system of record for application data; AWS is the deployment platform, not the data store.
- Destructive actions (e.g., cancelling an appointment or plan) require explicit user confirmation before being executed.

6. Other Requirements

No legal, regulatory, licensing, internationalization or reuse requirements beyond those already stated in Sections 1–5 have been identified for the current semester-project scope. The following items are noted for completeness and for possible future work:

- Internationalization/localization: the current scope targets English-language UI only; multilingual support is a possible future enhancement.
- Regulatory/legal: as an academic prototype, AyurSutra does not claim compliance with healthcare data-protection regulations (e.g., HIPAA/DPDP Act) for production clinical use; a compliance review would be required before any real clinical deployment.
- Licensing: all third-party libraries and frameworks shall be used in accordance with their respective open-source licenses.
- Reuse objectives: reusable UI components and API modules should be designed so they can be extended in a future capstone or production version of the project.

Appendix A: Glossary

Term	Definition
AI	Artificial Intelligence; assistive component used for personalized diet recommendations.
Admin	System-level user responsible for accounts, configurable therapies, analytics and reports.
Assessment	Structured practitioner-entered information about a Patient.
Doctor	Primary practitioner who manages care workflows and reviews AI recommendations.
Diet Plan	Structured meal/nutrition recommendation visible after Doctor approval.
FastAPI	Python web framework used to expose the AI service.
MongoDB Atlas	Cloud-hosted MongoDB service used for application data.
Nutrition Analysis	Analysis/comparison of available dietary and nutrient information.
Panchakarma	Ayurvedic therapeutic framework represented through configurable therapies and sessions.
Patient	End user who views their authorized care information.
Prakriti	An Ayurvedic assessment concept that may be recorded as practitioner-entered information.
RBAC	Role-Based Access Control.
Therapy Plan	Doctor-created plan connecting a Patient to a therapy and scheduled sessions.
Therapy Session	Individual scheduled occurrence within a Therapy Plan.
AI Recommendation Status	Lifecycle state such as Draft, Pending Review, Modified or Approved.

Acronyms

Acronym	Meaning
SRS	Software Requirements Specification
UML	Unified Modeling Language
ER	Entity-Relationship
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
AI	Artificial Intelligence

Acronym	Meaning
ML	Machine Learning
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
HTTPS	Hypertext Transfer Protocol Secure
UI	User Interface
UX	User Experience
MVP	Minimum Viable Product

Appendix B: Analysis Models

This appendix presents the textual specification of the system models and diagrams for AyurSutra. Their graphical renderings (use case, ER, architecture, activity and sequence diagrams) are to be produced by the project team using these definitions as the source specification and included in the final project report.

B.1 Use Case Model

Draw the system boundary as a large rectangle labelled 'AyurSutra'. Place Admin, Doctor and Patient outside the boundary. Place the AI Recommendation Service as an external service actor.

Admin Use Cases

- Login
- Manage Doctors
- Manage Patients
- Manage Therapies
- View Global Appointments
- View Analytics
- Generate Reports
- Manage Settings

Doctor Use Cases

- Login
- Search/Add Patient
- View Patient Profile
- Record Ayurvedic Assessment
- Record Relevant History
- Create Panchakarma Plan
- Schedule Therapy Session

- Manage Appointments
- Perform Nutrition Analysis
- Generate AI Diet Recommendation
- Review AI Recommendation
- Approve/Modify Diet Plan
- Record Progress
- Generate Reports

Patient Use Cases

- Login
- View Dashboard
- View Appointments
- View Therapies
- View Approved Diet Plan
- View Progress
- View Notifications
- Manage Profile

B.2 Entity-Relationship Model

The ER diagram should show the following entities, primary keys and foreign-key relationships.

Entity	Important Attributes / Relationships
User	user_id (PK), name, email, password_hash, role, status
Doctor	doctor_id (PK), user_id (FK), specialization, professional_id
Patient	patient_id (PK), user_id (FK), doctor_id (FK), profile/status
Assessment	assessment_id (PK), patient_id (FK), assessment fields, created_at
Therapy	therapy_id (PK), name, description/configuration, status
TherapyPlan	plan_id (PK), patient_id (FK), doctor_id (FK), therapy_id (FK), dates/status
TherapySession	session_id (PK), plan_id (FK), date/time, status, notes
Appointment	appointment_id (PK), patient_id (FK), doctor_id (FK), date/time, type/status
NutritionRecord	nutrition_id (PK), patient_id (FK), food/diet data, nutrients
DietPlan	diet_plan_id (PK), patient_id (FK), doctor_id (FK), source/status, meals, approval
ProgressRecord	progress_id (PK), patient_id (FK), doctor_id (FK), metrics, notes
Notification	notification_id (PK), user_id (FK), type, message, read_status
Report	report_id (PK), generated by (FK), type, parameters, created_at

Cardinalities

- One Doctor can manage many Patients.
- One Patient can have many Assessments, TherapyPlans, Appointments, NutritionRecords, DietPlans and ProgressRecords.
- One TherapyPlan belongs to one Patient, one Doctor and one Therapy.
- One TherapyPlan contains many TherapySessions.
- One User can receive many Notifications.
- A DietPlan may originate from AI and must store approval status.

B.3 System Architecture

Users (Admin, Doctor, Patient) interact with a React web application. The React frontend communicates with a Node.js + Express backend. The backend persists data to MongoDB Atlas and invokes a separate FastAPI AI Recommendation Engine. The full stack is hosted on AWS.

B.4 System Flow and Sequence

High-level flow:

- Login → Select Role → Admin / Doctor / Patient Dashboard.
- Doctor Care: Doctor → Patient → Assessment → Panchakarma Plan → Therapy Sessions → Appointments → Progress.
- AI Diet: Doctor → Patient Data → Nutrition Analysis → Backend → FastAPI AI Service → Recommendation → Doctor Review → Modify/Regenerate/Approve → Approved Diet Plan → Patient.
- Patient: Patient → Dashboard → Appointments → Therapies → Approved Diet Plan → Progress.

Recommended AI Recommendation Sequence

- Doctor selects Patient.
- Frontend requests authorized patient data.
- Backend validates authorization.
- Backend prepares recommendation input.
- Backend invokes FastAPI AI service.
- AI service returns structured recommendation.
- Backend stores recommendation as Pending Review.
- Doctor reviews and may modify/regenerate.
- Doctor approves final plan.
- Backend marks plan Approved.
- Patient retrieves and views approved plan.

B.5 Requirements Traceability Example

Artifact	Traceability Example
SRS	REQ-18: Doctor creates Panchakarma plan.
UI/UX	Create Therapy Plan screen and confirmation state.
API	Therapy Plan create endpoint.
Database	TherapyPlan collection/entity.
Test Case	TC-18: Verify authorized Doctor can create a valid plan.
Demo	Doctor login → Patient → Panchakarma → Create Plan.

The same traceability approach shall be applied to the AI recommendation requirements, particularly the rule that unapproved AI output must not become visible to Patients (REQ-41).

B.6 Supplementary Diagrams for the Final Report

- System Context Diagram
- Use Case Diagram
- ER Diagram
- System Architecture Diagram
- Activity Diagram for Patient Care Workflow
- Sequence Diagram for AI Diet Recommendation and Doctor Approval
- Flowchart for Role-Based Login
- Flowchart for Panchakarma Planning and Scheduling
- UI Sitemap / Navigation Diagram
- Requirements Traceability Matrix

Appendix C: To Be Determined List

The following items are not yet finalized and are tracked here as TBD items, each with an assigned reference for traceability.

TBD No.	Item	Related Section
TBD-01	Official registration numbers of all group members.	Title Page
TBD-02	Exact REST API route names and versioning scheme.	3.3 / 3.4
TBD-03	Final choice of AI/ML libraries and recommendation algorithm for the diet-recommendation engine.	2.4 / 4.7

TBD No.	Item	Related Section
TBD-04	Source and licensing of the nutrition/food dataset used for the academic prototype.	2.7 / 4.6
TBD-05	Final list of configurable Panchakarma therapies to be seeded in the catalog.	4.4
TBD-06	Final AWS deployment architecture (services, regions, scaling approach).	2.4
TBD-07	Whether a dedicated user manual/installation guide will be required as a separate deliverable.	2.6
TBD-08	Final wellness metrics to be tracked in Patient progress records.	4.8